第10章: アプリの公開（App Store / Google Play）

> いよいよ最終章です。完成したアプリを **App Store**（iPhone向け）と **Google Play**（Android向け）に公開し、世界中の人が使えるようにします。この章では、Expoのクラウドサービス **EAS** を使ったビルドと、各ストアへの申請手順を、完全初心者向けに解説します。

1.「公開」の全体像

スマホアプリの公開は、Webサイトの公開（第10章のNext.js版でやったVercelへのデプロイ）とは少し違い、**ストアの審査**を通る必要があります。流れを先に俯瞰しましょう。

① **開発者アカウントを登録**
Apple Developer（年$99）/ Google Play（初回$25）

↓

② **EASでビルド（アプリの完成品を作る）**
クラウドで .ipa（iOS）/ .aab（Android）を生成

↓

③ **ストアに提出（Submit）**
EASが各ストアへ自動アップロード

↓

④ **ストア情報の入力と審査**
説明文・スクショ・プライバシー設定 → 審査待ち

↓

🎉 **公開！**
あなたのアプリがストアに並びます

> **EAS（イーエーエス）とは？** Expo Application Services の略。Expoが提供する**クラウドのビルド・公開サービス**です。本来 iOSアプリのビルドにはMacとXcodeが必要ですが、EASを使えば**クラウド上のMacでビルド**してくれるため、**Windowsだけでも iOSアプリを作って公開できます**。これがExpoを選ぶ大きな理由の一つです。
>
> **費用について（重要）**: - **Google Play**: 開発者登録は **初回のみ $25**（一度きり）。- **Apple Developer**: **年間 $99**（毎年更新）。- **EAS自体**: 無料枠があり、個人開発の範囲なら基本無料で始められます（ビルド回数に制限あり）。
>
> 「とりあえず公開を体験したい」なら、費用が安く審査も比較的緩やかな **Google Play から始める**のがおすすめです。

---

2. 事前準備

2.1 Expoアカウントの作成とログイン

EASを使うにはExpoアカウントが必要です。まだなら https://expo.dev/ で無料登録します。その後、ターミナルからログインします。

コピー

```
npx expo login
# expo login : Expoアカウントにログインするコマンド
# メールアドレス（またはユーザー名）とパスワードを聞かれるので入力する
```

コピー

```
npx expo whoami
# whoami : 「今ログインしているのは誰か」を確認するコマンド
# 自分のユーザー名が表示されればログイン成功
```

2.2 EAS CLI の準備

コピー

```
npm install -g eas-cli
# npm install         : パッケージをインストール
# -g                  : global（グローバル）。PC全体で使えるコマンドとして入れる
# eas-cli             : EASを操作するコマンドラインツール
# これで eas というコマンドが使えるようになる
```

コピー

```
eas --version
# eas のバージョンを表示。番号が出ればインストール成功
```

2.3 プロジェクトをEASに紐付ける

コピー

```
eas init
# eas init : このプロジェクトをあなたのExpoアカウントのプロジェクトとして登録する
# 実行すると app.json に projectId（プロジェクトの識別子）が自動で追記される
```

---

3. ビルド設定 — `eas.json`

ビルド方法を定義する `eas.json` を作ります。次のコマンドで対話形式で生成できます。

コピー

```
eas build:configure
# build:configure : ビルド設定ファイル eas.json を生成する
# iOS / Android どちらをビルドするか聞かれたら「All（両方）」を選んでおくとよい
```

生成された `eas.json` の例と、各設定の意味です。

コピー

```
{
  "cli": { "version": ">= 5.0.0" },
  "build": {
    "development": {
      // 開発用ビルド（デバッグしやすい）
      "developmentClient": true,
      // 内部配布用（ストアには出さない）
      "distribution": "internal"
    },
    "preview": {
      // テスト配布用。実機に直接入れて試せる
      "distribution": "internal"
    },
    // 本番用（ストア提出用）
    "production": {
      // ビルドごとにビルド番号を自動で増やす
      "autoIncrement": true
    }
  },
  "submit": {
    // ストア提出の設定（後述）
    "production": {}
  }
}
```

3つのビルドプロファイル（development / preview / production）： - **development** … 開発中にデバッグ機能付きで動かすビルド。 - **preview** …「ストア提出前に、実機で最終確認する」ためのテスト配布ビルド。 - **production** … ストアに出す本番ビルド。

初心者はまず **preview** で実機テストし、問題なければ **production** でストア提出、という流れがおすすめです。

4. Android アプリを公開する（おすすめの最初の一歩）

4.1 まず preview ビルドで実機確認

コピー

```
eas build --profile preview --platform android
# eas build           : アプリをビルドする（クラウド上で実行される）
# --profile preview   : eas.jsonの「preview」設定を使う
# --platform android  : Android向けにビルド
# 実行するとクラウドでビルドが始まり、数分〜十数分で完了する
# 完了するとダウンロードURLが表示され、.apk を実機に入れて試せる
```

> **ビルドはクラウドで行われる:** `eas build` を実行すると、あなたのPCではなく**Expoのクラウドサーバー**でビルドが進みます。ターミナルにビルドの進捗URLが出るので、ブラウザで状況を見られます。完了を待つ間、PCで他の作業をしていてOKです。

4.2 本番ビルド（production）

実機確認で問題なければ、ストア提出用の本番ビルドを作ります。Androidのストア提出形式は `.aab`（**Android App Bundle**）です。

コピー

```
eas build --profile production --platform android
# --profile production : 本番設定でビルド。提出用の .aab ファイルが生成される
```

> **初回ビルド時の署名鍵:** 初めてビルドすると「アプリの署名鍵（Keystore）を生成してよいか」と聞かれます。**EASに管理を任せる（Yes）**を選べば、Expoが安全に鍵を生成・保管してくれます。この鍵は「このアプリは確かにあなたが作った」と証明する大切なものなので、EASに任せるのが初心者には安全です。

4.3 Google Play Console の準備

1. https://play.google.com/console/ で **Google Play デベロッパー登録**（初回$25）。
2. 「アプリを作成」でアプリ名（例: 書籍管理）・言語などを登録。
3. ストア掲載情報（説明文、スクリーンショット、アイコン、プライバシーポリシーのURLなど）を入力。

> **必要な掲載素材:** スクリーンショット（実機やエミュレータで撮影）、アプリの説明文、512×512のアイコン、フィーチャーグラフィック（1024×500）などが求められます。第9章で整えたアイコンが活きます。

4.4 EASでGoogle Playに提出

コピー

```
eas submit --profile production --platform android
# eas submit : ビルドしたアプリをストアにアップロードするコマンド
# 初回はGoogle Play APIの認証情報（サービスアカウントのJSON）を求められる
# → Google Play Consoleで発行し、案内に従って設定する
```

> **初回の認証設定:** 初めての提出時は、Google Play Consoleで「サービスアカウント」というアップロード用の認証情報（JSONファイル）を作る必要があります。手順はEASの案内とExpo公式ドキュメント（https://docs.expo.dev/submit/android/ ）に沿って進めれば完了します。一度設定すれば次回以降は自動です。

提出後、Google Play Console上で「製品版」リリースとして公開を申請します。審査は通常**数時間〜数日**です。

---

5. iOS アプリを公開する

5.1 Apple Developer Program への登録

iOSの公開には **Apple Developer Program（年$99）** の登録が必須です。
https://developer.apple.com/programs/ から登録します（Apple IDが必要）。

> **Macが無くても大丈夫:** 前述の通り、EASがクラウドのMacでビルドするため、**Windowsだけでも iOSアプリをビルド・提出できます。**ただしApple Developerの登録手続き自体はブラウザで行います。

5.2 ビルドと提出

コピー

```
# まずpreviewで確認（任意。TestFlightで実機確認する方法もある）
eas build --profile production --platform ios
# --platform ios : iOS向けにビルド。提出用の .ipa ファイルが生成される
# 初回はApple IDでのログインや、署名証明書(Certificate)の生成をEASが案内してくれる
```

> **iOSの署名（証明書とプロビジョニング）:** iOSは署名の仕組みが複雑ですが、EASが**対話形式で自動生成**してくれます。「Appleアカウントでログイン → EASに証明書管理を任せる」と進めれば、難しい設定を意識せずに済みます。

コピー

```
eas submit --profile production --platform ios
# iOSアプリを App Store Connect にアップロードする
# 初回はApp Store Connectでアプリ枠を作成しておく必要がある
```

5.3 App Store Connect での申請

1. https://appstoreconnect.apple.com/ でアプリ情報（名前・説明・スクリーンショット・カテゴリ・年齢制限など）を入力。
2. プライバシー情報（どんなデータを集めるか）を申告。
3. 「審査に提出」を押す。

> **iOSの審査は厳しめ:** Appleの審査はGoogleより厳格で、**数日**かかることが多いです。リジェクト（却下）されることもありますが、理由が通知されるので、それに従って修正・再提出すれば大丈夫です。よくある却下理由は「説明不足」「クラッシュ」「プライバシー申告漏れ」などです。

---

6. 公開後のアップデート

6.1 アプリを更新する2つの方法

公開後にバグ修正や機能追加をする方法は2つあります。

方法	何ができるか	再審査
EAS Update（OTA更新）	JS/TSのコード変更を、ストアを通さず即時配信	不要（条件あり）
新バージョンのビルド＋提出	ネイティブ部分も含む全面更新	必要

6.2 EAS Update（OTA更新）

EAS Update は「Over The Air（OTA、無線経由）更新」とも呼ばれ、**JavaScript/TypeScriptの変更だけなら、ストア審査を経ずにユーザーへ即配信**できる仕組みです。

コピー

```
npx expo install expo-updates
# expo-updates : OTA更新を受け取る機能をアプリに追加する部品

eas update --branch production --message "誤字を修正"
# eas update           : OTA更新を配信するコマンド
# --branch production  : どの配信ラインに出すか
# --message "..."      : 更新内容のメモ（あとで見返す用）
# 実行すると、対象アプリが次回起動時に新しいコードを取得する
```

> **OTA更新の注意:**「文言の修正」「色の変更」「ロジックの修正」などJS/TSの範囲はOTAで即配信できますが、**新しいネイティブ機能の追加（カメラ権限を増やす等）はOTAでは反映できず、再ビルド＆再提出が必要**です。また、ストアの規約上「アプリの本質を変える」更新はOTAで行わずビルド提出すべきとされています。小さな修正＝OTA、大きな変更＝再ビルド、と覚えましょう。

6.3 バージョンを上げて再提出

機能追加などでは、`app.json` の `version`（例: `1.0.0` → `1.1.0`）を上げ、再度 `eas build` → `eas submit` します。`eas.json` の `autoIncrement: true` により、ビルド番号は自動で増えます。

---

7. 公開前の最終チェックリスト

公開申請の前に、以下を確認しましょう。

- [ ] アプリが実機（preview ビルド）でクラッシュせず正常に動く
- [ ] **Supabaseの本番設定**: 第5章で「全員OK」にしたRLSを、適切に制限したか（重要）
- [ ] `.env` の接続情報が正しく、**`service_role` キーを含めていない**
- [ ] アプリ名・アイコン・スプラッシュが設定済み（第9章）
- [ ] `app.json` の `android.package` / iOSのbundle identifierが固有の値になっている
- [ ] スクリーンショット・説明文・**プライバシーポリシー**を用意した
- [ ] 年齢制限・カテゴリなどストア情報を入力した

> **セキュリティの再確認（最重要）**：第5章でRLSを「全員が全操作OK」に設定しました。これは**学習用の設定**です。実際に公開するアプリでは、悪意ある人にデータを書き換えられないよう、**必ず適切なポリシーに変更**してください。たとえば「読み取りは全員OK、書き込みはログインユーザーのみ」など。認証機能を入れる場合はSupabase Authと組み合わせます（本書の範囲を超えるため、Expo/Supabase公式ドキュメントを参照）。

---

8. この章のまとめ

- スマホアプリの公開は「**開発者登録 → EASビルド → ストア提出 → 審査 → 公開**」の流れ
- **EAS** を使えば、**WindowsだけでもiOS/Android両方**のアプリをクラウドでビルド・提出できる
- 費用: Google Play（初回$25）、Apple Developer（年$99）。**まずはGoogle Playから**がおすすめ
- `eas build`（ビルド）→ `eas submit`（提出）。署名鍵や証明書はEASに任せられる
- 公開後は **EAS Update（OTA）** で小さな修正を即配信、大きな変更は再ビルド＆再提出
- 公開前に**RLSの本番設定**など、セキュリティを必ず見直す

---

9. おわりに — おめでとうございます！ 🎉

ここまで完走したあなたは、

- TypeScript / React / React Native の基礎
- Expo を使ったアプリ開発
- Supabase によるデータ管理（CRUD）
- NativeWind によるスタイリング
- EAS を使ったストアへの公開

という、**スマホアプリ開発の全工程**を体験しました。これは「アイデアを形にして世界に届ける」力そのものです。

> **次のステップ**: - **認証機能**: Supabase Auth でログイン機能を追加し、「自分の本棚」を実現する - **プッシュ通知**: `expo-notifications` で通知を送る - **他の機能**: カメラでバーコードを読んで本を登録、など - **Web版との連携**: 姉妹編 `next` のWebアプリと同じSupabaseを共有し、マルチプラットフォーム化
>
> そして、次の第11章では **Claude Code を使った「バイブコーディング」** で、AIと一緒にさらに効率よく開発を進める方法を紹介します。本当にお疲れさまでした！